

Extracting decimal digits of π in quasi-linear time from binary BBP formulas

Joseph Babčanec
Benedict College
`joseph.babcanec@benedict.edu`

July 2026

Abstract

The Bailey–Borwein–Plouffe (BBP) formula computes isolated binary or hexadecimal digits of π without computing earlier digits, but no analogous decimal formula exists: Borwein, Borwein and Galway proved that π admits no Machin-type BBP arctangent formula in any base that is not a power of 2, and every published decimal digit-extraction algorithm runs in quadratic time or worse. We show that the N -th decimal digit of π can nevertheless be computed from the ordinary hexadecimal BBP formula in quasi-linear time — $O(N \log^3 N)$ bit operations — at the price of $\Theta(N)$ bits of read-only auxiliary space holding the binary expansion of the single integer 5^{N-1} : after multiplying the BBP series termwise by 10^{N-1} , every term takes the form $\pm 2^E 5^{N-1} / \mu$ with μ odd; terms with $E \geq 0$ reduce to word-size modular exponentiation, while the $\Theta(N)$ terms with $E < 0$ satisfy an exact identity that expresses their fractional parts through residues and bit-windows of the *shared* integer 5^{N-1} , batchable with accumulating remainder trees. The method applies verbatim to every constant with a BBP formula in a base 2^r (e.g. $\ln 2$), yields the digits of π in *any* base B in $\tilde{O}(N)$ time, and resolves the obstruction described by Zudilin (arXiv:2409.10097), whose base-5 (originally base-10) attempt fails precisely on the $E < 0$ terms. Implementations in Python and C++ are verified at decimal positions through 10^7 against independent computations and public digit corpora. We then reduce every phase but one to $O(N \log \log N)$ word operations: sieving the modulus progressions turns all modular exponentiations into geometric progressions along prime-indexed subprogressions, with $O(1)$ starters by Euler’s criterion — an amortization that applies verbatim to classical binary BBP extraction — while a Bézout split makes half of every remaining term word arithmetic and the other halves consolidate into a *single* 2-adic integer V with a division-free construction. A GMP-backed implementation of this refined pipeline measures a fitted exponent of $N^{1.08}$ across positions 10^4 – 10^7 — the quasi-linear regime in practice, thirty times faster at 10^7 than a multithreaded implementation of the unbatched algorithm. The one remaining logarithm is analyzed as a barrier with seven machine-validated faces — merge depth, rising-factorial products, the discrete-logarithm sum $E = \sum_{k \leq K} \text{dlog}_3(1 + 8k)$, a 2-adically *convergent* Euler–Maclaurin identity linking E to Diamond’s 2-adic log-gamma function, and an elementary huge-exponent power sum — and a classical factorization technique of Borwein then splits them: the products, discrete-logarithm sums, power sums, and log-gamma values all fall to $O(N \log^2 N)$ (machine-validated), while the digit pipeline’s additive core — one consolidated sum of scaled 2-adic inverses — remains at the product-tree bound and is isolated as the precise open problem. No factoring-hardness argument protects it. We state the conjectural $N \log N$ floor precisely, together with the sublinear-time and polylogarithmic-space questions that remain open.

1 Introduction

Write $d_N(\xi)$ for the N -th decimal digit of $\xi > 0$ after the decimal point, so that with $\{x\} = x - \lfloor x \rfloor$,

$$d_N(\xi) = \lfloor 10 \{10^{N-1} \xi\} \rfloor. \quad (1)$$

The *digit extraction problem* asks for $d_N(\pi)$ without computing the $N-1$ digits before it. Bailey, Borwein and Plouffe [1] solved the binary case with the formula

$$\pi = \sum_{k=0}^{\infty} \frac{1}{16^k} \left(\frac{4}{8k+1} - \frac{2}{8k+4} - \frac{1}{8k+5} - \frac{1}{8k+6} \right), \quad (2)$$

which yields hexadecimal digits at position N in $O(N \log N)$ -type time and $O(\log N)$ space via per-term modular exponentiation. For base 10 no such formula exists: Borwein, Borwein and Galway [2] proved that π has no Machin-type BBP arctangent formula in any base $b \neq 2^m$. The known decimal algorithms form a ladder of quadratic-or-worse methods: Plouffe’s original scheme [3] runs in $O(N^3 \log^3 N)$ time, Bellard’s improvement [4] in $O(N^2)$ time in any base, and Gourdon’s method [5] in $O(N^2 \log \log N / \log^2 N)$ word operations with $O(\log^2 N)$ memory. Plouffe’s 2022 approach [6] extracts a digit from an asymptotic expansion whose coefficients are Bernoulli (or Euler) numbers; it requires precomputed tables of those numbers and is thereby capped near position 10^8 , the current extent of Bernoulli computations. Most recently Zudilin [7] attempted a BBP-style computation of π in base 5 — the first arXiv version was titled “...in base 10” — via a Salikhov-type representation over $\mathbb{Z}[i]$, and identified an “obvious flaw”: his per-term modular identity fails exactly on the terms in which the power of 5 in the numerator is exhausted, and no truncation avoids them. He left the repair as an open question; the note has no published successor.

This paper repairs that flaw — not for Zudilin’s Gaussian series specifically, but for the ordinary formula (2) and, with it, every binary BBP constant. The price is memory: we abandon the $O(\log N)$ -space regime of BBP and store the binary expansion of the single integer $X = 5^{N-1}$, about $0.2902 N$ bytes. In exchange the running time becomes quasi-linear.

Theorem 1 (main). *Let $M(n)$ denote the cost of multiplying two n -bit integers. The N -th decimal digit of π (together with any constant window of following digits) can be computed, without computing any earlier digit, in*

$$O(M(N) + M(N \log N) \log N + N \log^2 N \log \log N) \subseteq O(N \log^3 N)$$

bit operations, using $\Theta(N)$ bits of auxiliary space of which all but $O(\log^2 N)$ bits are a read-only string shared by all terms, namely the binary expansion of 5^{N-1} . The output is certified in the Las Vegas sense of Remark 6.

The same statement holds with π replaced by any constant possessing a BBP-type formula in a base 2^r (Theorem 14), for instance $\ln 2$; and with “decimal” replaced by “base B ” for arbitrary $B \geq 2$ (Corollary 15), which supersedes the $O(N^2)$ any-base algorithm of [4] in time at the stated memory cost. Section 4 then sharpens the algorithm structurally: all phases except one collapse to $O(N \log \log N)$ word operations, and the surviving phase — the construction of a single 2-adic integer — is analyzed as a barrier in Section 6.

What is, and is not, being claimed. Computing *all* N digits of π by binary splitting of the Chudnovsky series already takes softly linear time, so quasi-linear cost per isolated digit is not by itself a new capability; and the canonical open problem posed in [1] — decimal digit extraction in polylogarithmic (or even sublinear) space — remains open and is *not* resolved here, since our space is $\Theta(N)$ bits. The contributions are: (i) the first digit-extraction algorithm for decimal π based on a BBP formula, with per-term independence: after the one shared pre-computation the terms are processed in any order by any number of workers that communicate only word-size results, in the style of distributed binary-BBP projects; (ii) to our knowledge the first *implemented and verified* subquadratic decimal extractor in any model (Section 7); (iii) the resolution of the specific obstruction of [7]; (iv) an auxiliary object — one power of 5 computed on the fly — rather than tables that must be precomputed by other means, in contrast to [6]; (v) the sieve-batched exponentiation of Section 4, which also accelerates classical binary BBP extraction; and (vi) a precise, machine-validated characterization of the obstruction to going faster (Section 6). Closest to (ii) at the level of results, Gourdon’s unpublished 2003 manuscript [5] asserts in its final section a time–memory tradeoff, $O(n^2 \log^3 n \log \log n / (m \log^2(n/m)))$ time at memory $m \in [n^\varepsilon, n]$, whose endpoint $m \approx n$ would likewise be softly linear; the manuscript states that “the details of the algorithm will be added soon in a next version,” but no later version ever appeared (the archived file is unchanged from February 2003 through 2026), no details or implementation are known, and the sketch that is given (binary splitting over blocks) differs structurally from the present method. We therefore claim the algorithm and its analysis, not priority over the bare assertion that some such tradeoff should exist.

2 The three-phase decomposition

Throughout, $N \geq 1$ is the target position, $n := N - 1$, $X := 5^n$, and $L := \lceil n \log_2 5 \rceil$ the bit length of X . Since $x \mapsto \{x\}$ is a homomorphism $\mathbb{R} \rightarrow \mathbb{R}/\mathbb{Z}$, fractional parts of individual terms may be accumulated with signs modulo 1; alternating or negative coefficients pose no obstacle.

2.1 Normal form

Multiply (2) termwise by $10^n = 2^n 5^n$ and absorb all powers of two — the numerators 4, 2, 1, 1 and the even parts of $8k + 4 = 4(2k + 1)$ and $8k + 6 = 2(4k + 3)$ — into a single exponent. With $A := n - 4k$:

Lemma 2 (normal form).

$$10^n \pi = \sum_{k \geq 0} \left[2^{A+2} \frac{X}{8k+1} - 2^{A-1} \frac{X}{2k+1} - 2^A \frac{X}{8k+5} - 2^{A-1} \frac{X}{4k+3} \right], \quad (3)$$

i.e. every term equals $\pm 2^E X/\mu$ with μ odd and $E \in \{A+2, A-1, A, A-1\}$.

Proof. Immediate from $10^n 16^{-k} = 2^{n-4k} 5^n$ and the factorizations $8k+4 = 2^2(2k+1)$, $8k+6 = 2(4k+3)$. $\square$

Terms decay like 16^{-k} , so positions N through $N + O(1)$ are determined by the terms with $k \leq K$, where $K = \lceil (n \log_2 10 + G + c)/4 \rceil$ for a G -bit accumulator (Section 2.5); in total $4(K+1) \approx 3.33 N$ terms. Each term falls into one of three regimes.

2.2 Phase 1: easy terms ($E \geq 0$)

Here $2^E X$ is a nonnegative integer, so

$$\left\{ \frac{2^E X}{\mu} \right\} = \frac{(2^E \bmod \mu)(5^n \bmod \mu) \bmod \mu}{\mu},$$

computable with two modular exponentiations modulo the word-size integer μ — exactly as in classical BBP. There are $\Theta(N)$ such terms ($E \geq 0$ holds for $k \lesssim n/4$ in each family).

2.3 Phase 2: band terms ($E < 0$, $J := -E < L$)

These are the terms on which every previous decimal approach loses: the numerator is no longer an integer, the natural modulus $\mu 2^J$ grows exponentially along the family, and per-term reduction of X modulo $\mu 2^J$ costs $\Theta(M(N))$, giving $\Theta(N)$ -many big operations — the quadratic wall, and precisely the failure identified in [7, §3]. The wall disappears because *all band terms share the same numerator X* :

Lemma 3 (band identity). *Let $X, \mu \geq 1$ and $J \geq 1$ be integers, $H := \lfloor X/2^J \rfloor$, $h := H \bmod \mu$, and $u := \{X/2^J\} \in [0, 1)$. Then*

$$\left\{ \frac{X}{\mu 2^J} \right\} = \frac{h + u}{\mu}.$$

Proof. $X/(\mu 2^J) = (H + u)/\mu$. Writing $H = q\mu + h$, $X/(\mu 2^J) = q + (h + u)/\mu$ with $0 \leq (h + u)/\mu < (\mu - 1 + 1)/\mu = 1$, so $(h + u)/\mu$ is the fractional part. $\square$

Lemma 4 (recovering h). *Let μ be odd, $R := X \bmod \mu$ and $\tilde{Y} := (X \bmod 2^J) \bmod \mu$. Then*

$$h = (R - \tilde{Y}) \cdot 2^{-J} \bmod \mu.$$

Proof. From $X = 2^J H + (X \bmod 2^J)$, reduce modulo μ and use that 2 is invertible mod μ . $\square$

Consequently a band term needs: $R = 5^n \bmod \mu$ and $2^{-J} \bmod \mu = ((\mu + 1)/2)^J \bmod \mu$, two word-size modular exponentiations; the *prefix residue* $\tilde{Y}$, discussed in Section 3.2; and u to G_p bits, which is by definition the bit-window of X at depth J :

$$\lfloor 2^{G_p} u \rfloor = \left\lfloor \frac{X \bmod 2^J}{2^{J-G_p}} \right\rfloor = \text{bits } [J - G_p, J) \text{ of } X,$$

an $O(1)$ read from the stored binary expansion of X . This is the entire role of the $\Theta(N)$ -bit auxiliary string: it converts every band term's “impossible” modulus $\mu 2^J$ into one shared object read locally.

2.4 Phase 3: tail terms ($J \geq L$)

Then $H = 0$, so the term is u/μ with $u = X/2^J < 1$ read from the top of the stored string; only $O(G)$ terms lie between $J = L$ and the accumulator cutoff.

2.5 Accumulation and error

All contributions are accumulated in a fixed-point register of G fraction bits, each term entering as $\lfloor 2^G(h+u)/\mu \rfloor$ (band/tail) or $\lfloor 2^G r/\mu \rfloor$ (easy), with sign, modulo 2^G .

Proposition 5 (error bound). *With window precision G_p and T terms processed, the accumulator differs from $2^G\{10^n\pi\}$ by at most $T(1+2^{G-G_p})+2$ units, i.e. by $T \cdot 2^{-G} + T \cdot 2^{-G_p} + O(2^{-G})$ after rescaling. Choosing $G_p, G = \Theta(\log N)$ with $G_p \geq \log_2 T + g$ and $G \geq G_p + 48$ leaves g trustworthy guard bits above the output window.*

Proof. Each floor loses less than one unit in the last place; truncating u to G_p bits perturbs $(h+u)/\mu$ by less than $2^{-G_p}/\mu \leq 2^{-G_p}$; the series cutoff contributes less than one unit by the choice of K . Sum over T terms. $\square$

Remark 6 (certification). As with all BBP-type extraction, the digit window is read off the top of the accumulator and is provably correct unless $\{10^n\pi\}$ lies within the total error ε of a carry boundary of the window — i.e. unless the digits following the window form a run of 9s or 0s of length $\approx (\log_{10} \varepsilon^{-1}) - w$. The algorithm detects this condition in the output and re-runs with doubled G ; under the standard heuristic that the digits of π are equidistributed the expected number of passes is $1 + o(1)$. (An unconditional guard follows from the irrationality measure $\mu(\pi) \leq 7.11$ of Zeilberger–Zudilin [15], but only with $G = \Theta(N)$, which would sacrifice quasi-linearity; every implementation of binary BBP makes the same tradeoff.)

3 Complexity

3.1 Precomputation and per-term costs

$X = 5^n$ is computed by repeated squaring in $O(M(N))$ bit operations — the $O(\log N)$ squarings act on geometrically growing operands, so the cost telescopes — and stored as $\approx 0.2902N$ bytes. The $O(N)$ modular exponentiations (Phases 1–2) cost $O(\log N)$ multiplications of $O(\log N)$ -bit words each: $O(N \log N)$ word operations, i.e. $O(N \log^2 N \log \log N)$ bit operations; window reads are $O(1)$ words each.

3.2 Batched prefix residues

The one nontrivial cost is the family of prefix residues $\tilde{Y}_t = (X \bmod 2^{J_t}) \bmod \mu_t$ over the $T = \Theta(N)$ band terms. Computed naively (a Horner pass over the stored string per term) each costs $O(J_t/w)$ word operations — an $O(N^2/w)$ total that our v1 implementations accept (Section 7). It is however batchable with remainder-tree technology [9, 10]:

Theorem 7 (prefix-residue batching). *Given the binary expansion of X (of length L) and pairs $(J_1, \mu_1), \dots, (J_T, \mu_T)$ with word-size moduli and the J_t in arithmetic progression, all residues $(X \bmod 2^{J_t}) \bmod \mu_t$ can be computed in $O(M(L + Tw) \log T)$ bit operations.*

Proof sketch. Let δ be the common difference of the J_t (for the four families of (3), $\delta = 4$; run one instance per family) and let $c_t \in [0, 2^\delta)$ be the bits of X in $[J_t, J_{t+1})$. The prefix values $P_t := X \bmod 2^{J_t}$ obey the linear recurrence $P_{t+1} = P_t + 2^{J_t} c_t$, i.e. the pair $v_t = (P_t, 2^{J_t})^\top$ transforms as $v_{t+1} = A_t v_t$ with the word-size upper-triangular matrix

$$A_t = \begin{pmatrix} 1 & c_t \\ 0 & 2^\delta \end{pmatrix}, \quad \text{so} \quad \tilde{Y}_t = [(A_{t-1} \cdots A_0) v_0]_1 \bmod \mu_t.$$

Computing a running product of word-size matrices reduced modulo a different word-size modulus at each leaf is precisely the *accumulating remainder tree* of Costa–Gerbic–Harvey and Harvey–Sutherland [11, 12]: one product/remainder tree over the T leaves in which the partial products (which grow to $\Theta(L)$ bits at the root) are pushed down and reduced modulo the subtree modulus products (total size $\Theta(Tw)$). Each of the $O(\log T)$ levels costs $O(M(L + Tw))$ by superadditivity of M , giving the bound. The workspace is $O(L + Tw)$ bits, processed level by level. $\square$

With $L = \Theta(N)$, $T = \Theta(N)$, $w = O(\log N)$ this is $O(M(N \log N) \log N) \subseteq O(N \log^3 N)$, and Theorem 1 follows by summing the phase costs: the accumulating tree dominates, the modular exponentiations contribute $O(N \log^2 N \log \log N)$ bits, and everything else is smaller. In the word RAM the totals are $O(N \log^2 N)$ word operations for the tree and $O(N \log N)$ for the exponentiations — and Section 4 reduces the latter, and every other non-tree cost, to $O(N \log \log N)$.

3.3 Comparison

Method	Time	Space	Base	Status
BBP 1997 [1]	$\tilde{O}(N)$	$O(\log N)$	2^r only	implemented
Plouffe 1996 [3]	$O(N^3 \log^3 N)$	$O(\log N)$	any	implemented
Bellard 1997 [4]	$O(N^2)$	$O(\log N)$	any	implemented
Gourdon 2003 [5], Thm. 1	$O(N^2 \log \log N / \log^2 N)^a$	$O(\log^2 N)$	10	implemented
Gourdon 2003 [5], Thm. 2	$O(\frac{N^2 \log^3 N \log \log N}{m \log^2(N/m)})$	$O(m)$	10	asserted only
Plouffe 2022 [6]	(no analysis; $N \leq 10^8$)	tables B_{2n}	10, 2	implemented
This paper	$O(N \log^3 N)$	$\Theta(N)$ bits (shared, r/o)	any	implemented ^b

^aword operations. ^bmeasured $N^{1.077}$ over $N \in [10^4, 10^7]$ by the refined pipeline (v3); see Section 7.

4 Faster phases: sieve-batching and a 2-adic normal form

4.1 Sieve-batched exponentiation

The exponentiation workload of Sections 2–3 consists, per term, of powers of 2 whose exponents are *linear in the index* k (namely $2^{\pm 4k}$ -type factors and 2^{-J} with J linear in k) and powers with *fixed* exponents (5^n and $2^{n+O(1)}$ modulo μ). Square-and-multiply prices each at $O(\log N)$ word multiplications: $O(N \log N)$ in total. Both kinds amortize.

Lemma 8 (Euler starters). *Let q be an odd prime and k_0 minimal with $q \mid 8k_0 + 1$, and write $8k_0 + 1 = qs$; then $s \leq 7$ is odd and*

$$2^{4k_0} \equiv \left(\frac{2}{q}\right)^s 2^{(s-1)/2} \pmod{q},$$

computable in $O(1)$ word operations. Analogous $O(1)$ formulas hold for the progressions $8k + 5$, $4k + 3$, $2k + 1$ (using $2^{4s-3} \equiv 2^{s-3}$ etc.).

Proof. $k_0 < q$ gives $s \leq 7$; qs odd gives s odd. Then $4k_0 = (qs - 1)/2 = s \cdot \frac{q-1}{2} + \frac{s-1}{2}$, and $2^{(q-1)/2} \equiv (\frac{2}{q}) \pmod{q}$ is Euler’s criterion; the Legendre symbol is read off $q \bmod 8$. $\square$

Lemma 9 (Fermat chain constants). *For an odd prime q , $2^{4q} \equiv 2^4 = 16 \pmod{q}$, and likewise $2^{-4q} \equiv 16^{-1} \pmod{q}$.*

Theorem 10 (sieve-batched exponentiation). *All 2-power residues with k -linear exponents, over all terms of (3) with $k \leq K$, are computable in $O(K \log \log K)$ word operations; all fixed-exponent residues $(5^n \bmod \mu, 2^{n+O(1)} \bmod \mu)$ likewise in $O(K \log \log K)$ word operations, the per-term cost of the once-per-prime-power computations vanishing as $O(1/\log K)$. Hence the entire exponentiation phase costs $O(N \log \log N)$ word operations.*

Proof sketch. A segmented sieve over each progression delivers the factorization of every μ at $O(K \log \log K)$ total cost. Fix a prime power $P = q^e$ dividing some moduli; the indices k with $P \mid \mu(k)$ form an arithmetic progression mod P , along which $2^{4k} \bmod P$ is a geometric sequence with ratio $2^{4P} \bmod P$: by Lemmas 8–9 the starter and (for $e = 1$) the ratio are $O(1)$, and each further element costs one multiplication. The number of (term, prime-power) incidences is $\sum_{k \leq K} \Omega'(\mu(k)) = O(K \log \log K)$ by Mertens; prime powers with $e \geq 2$ have $O(\sum_q K/q^2) = O(K)$ incidences and afford honest exponentiations for their rare starters. Fixed-exponent residues are computed once per *distinct* prime power — there are $O(K/\log K)$ of them, at $O(\log N)$ each — and every term’s residue is assembled from its $O(\log \log K)$ cached prime-power components by CRT at $O(1)$ multiplications per incidence. $\square$

Remark 11 (application to binary BBP). The classical BBP loop computes $16^{d-k} \bmod (8k+j)$ per term by square-and-multiply, and has since 1997. The structure required by Theorem 10 — fixed base, exponent linear in k , modulus on an arithmetic progression in k — is exactly the structure of that loop, so the same sieve-batching applies to hexadecimal/binary digit extraction of π and to every BBP-type computation. We are not aware of a prior appearance of this amortization. Our validation run (Section 7) measures $2.7\times$ fewer multiplications already at $K = 2 \cdot 10^4$; the ratio scales as $\Theta(\log K / \log \log K)$.

4.2 The Bézout split and the 2-adic normal form

Lemma 12 (Bézout split). *Let μ be odd, $J \geq 1$, $u = 2^{-J} \bmod \mu$, and $w = \mu^{-1} \bmod 2^J$. Then, exactly,*

$$\left\{ \frac{X}{\mu 2^J} \right\} = \left\{ \frac{(X \bmod \mu) u \bmod \mu}{\mu} + \frac{Xw \bmod 2^J}{2^J} \right\}.$$

Proof. With $v = (1 - u2^J)/\mu \in \mathbb{Z}$ we have $u2^J + v\mu = 1$, hence $X/(\mu 2^J) = Xu/\mu + Xv/2^J$; reduce each part mod 1 and note $v \equiv \mu^{-1} \pmod{2^J}$. $\square$

The first summand is word arithmetic, and its ingredients $X \bmod \mu$ and $u = 2^{-J} \bmod \mu$ are exactly what Theorem 10 delivers at amortized $O(\log \log N)$: *the odd halves of all band terms join the sieved word phase.* The dyadic halves collapse:

Lemma 13 (dyadic consolidation). *Let $J_{\max} = \max_t J_t$ and*

$$V = \sum_t \pm 2^{J_{\max} - J_t} (\mu_t^{-1} \bmod 2^{J_{\max}}) \bmod 2^{J_{\max}}.$$

Then $\sum_t \pm \{X(\mu_t^{-1} \bmod 2^{J_t})/2^{J_t}\} \equiv \{XV/2^{J_{\max}}\} \pmod{1}$.

Proof. Write $\mu^{-1} \bmod 2^{J_{\max}} = (\mu^{-1} \bmod 2^J) + 2^J t$; then $X(\mu^{-1} \bmod 2^{J_{\max}})/2^J = X(\mu^{-1} \bmod 2^J)/2^J + Xt$ and the integer Xt vanishes mod 1. Summing and factoring $2^{-J_{\max}}$ gives the claim; replacing V by $V \bmod 2^{J_{\max}}$ changes the value by an integer multiple of X . $\square$

The refined algorithm. The extractor is thereby restructured as:

1. **Sieve phase** (Theorem 10): factorizations, chains, caches — $O(N \log \log N)$ word operations.
2. **Word phase**: all easy terms and all odd halves of band terms accumulate in fixed point — $O(N \log \log N)$ word operations.
3. **V phase**: build V by a balanced merge of (numerator, denominator-product) pairs *modulo powers of two*: node arithmetic is truncated multiplication only, and one final Newton–Hensel inversion mod $2^{J_{\max}}$ recovers V — cost $O(M(N \log N) \log N)$, division-free. (The exact-sum form $X \cdot (A/B)$ with $A/B = \sum_t \pm 1/(\mu_t 2^{J_t})$ built by binary splitting is an equivalent second variant with the same asymptotics.)
4. **Finish**: one product $XV \bmod 2^{J_{\max}}$, one window read, digit output — $O(M(N))$.

Every logarithm in Theorem 1 now lives in step 3.

5 Generalizations

Theorem 14. *Let ξ admit a BBP-type formula in base 2^r , $\xi = \sum_{k \geq 0} 2^{-rk} p(k)/q(k)$ with integer polynomials and q having word-size values, in the sense of [1]. Then the N -th decimal digit of ξ is computable in the time and space of Theorem 1.*

Proof. Termwise, $10^n 2^{-rk} p(k)/q(k) = 2^{n-rk} 5^n p(k)/q(k)$; folding the 2-part of $q(k)$ into the exponent yields the normal form $\pm 2^E 5^n p(k)/\mu$ with μ odd, and Lemmas 3–4 apply with numerator $p(k)X$ (h and u scale by the word-size $p(k)$ before the fixed-point division). All counts are as before. $\square$

For example $\ln 2 = \sum_{k \geq 1} 1/(k 2^k)$ gives, with $k = 2^s \mu$, terms $2^{n-k-s} 5^n/\mu$: the first quasi-linear-time decimal digit extractor for $\ln 2$. The same holds for π^2 , $\ln^2 2$, Catalan-type combinations, and the rest of the base- 2^r BBP zoo of [8].

Corollary 15 (any base). *For any base $B \geq 2$, the N -th base- B digit of any base- 2^r BBP constant is computable in $\tilde{O}(N)$ time with $\Theta(N \log B)$ bits of read-only auxiliary space, namely the expansion of Q^{N-1} where Q is the odd part of B .*

Proof. $B^n = 2^{an} Q^n$ with Q odd; the normal form becomes $\pm 2^E Q^n/\mu$ and the argument is unchanged with $X = Q^n$. (For odd B , $a = 0$ and every non-easy term is band or tail.) $\square$

In particular the base-5 digits of π are computable in quasi-linear time with a shared string $X = 5^{N-1}$ — in that case *every* term with $k \geq 1$ is a band or tail term. This answers the question left open in [7] affirmatively at $\Theta(N)$ -bit space: the terms on which his identity (3) fails (denominator $5^{2n+1-d+k} m$) are exactly our band terms, and Lemma 3 computes them. Whether polylogarithmic space suffices remains open, and our reliance on stored middle bits of 5^n suggests a connection to the apparent hardness of computing isolated middle bits of integer powers.

We also note that the framework accepts any Machin-type formula whose arctangent arguments are 2-5-smooth, such as the 1844 Strassnitzky–Dase identity $\pi/4 = \arctan \frac{1}{2} + \arctan \frac{1}{5} + \arctan \frac{1}{8}$ (equivalently $(2+i)(5+i)(8+i) = 65(1+i)$ in $\mathbb{Z}[i]$); by Størmer-type finiteness these smooth identities are scarce, and we use this one only as an independent correctness check.

6 The barrier

After Section 4, every phase except the construction of V runs in $O(N \log \log N)$ word operations; the V phase remains at the product-tree bound. This section characterizes the obstruction.

6.1 Status of lower bounds

No unconditional lower bound is available at any level: proving even $\Omega(N)$ for the digit itself would require ruling out closed forms for the digits of π , far beyond current techniques — an obstacle shared with binary BBP. Within the framework, $\Omega(N)$ is forced structurally: $\Theta(N)$ terms each contribute at $O(1)$ magnitude modulo 1, and the u -windows jointly tile $\Theta(N)$ bits of X . The natural conjectural floor is $N \log N$: every known route to the string of $X = 5^{N-1}$ costs $\Theta(M(N))$, and $M(N) = \Theta(N \log N)$ is believed optimal — the upper bound is Harvey–van der Hoeven [13], and a matching lower bound is known only conditionally, on a network-coding conjecture [14]; no reduction from multiplication to fixed-base powering is known, so even that conditional bound does not formally transfer. Sublinear time would require abandoning the series paradigm altogether, since every term matters at full weight modulo 1 — and three decades of work on binary BBP have not produced $o(N)$ extraction in any base. Everything between the proven $O(N \log^3 N)$ and the conjectural $N \log N$ floor is the subject of what follows.

6.2 Equivalent faces

The denominator product entering the V phase is, per family, a rising-factorial-type product: e.g. $\prod_{k \leq K} (8k + 1) = 8^{K+1} (1/8)^{\overline{K+1}}$. Computing such products (or V , which every known merge scheme routes through them) modulo $2^{\Theta(N)}$ below the product-tree bound $M(n \log n) \log n$ is precisely the classical factorial barrier: no algorithm is known to compute $n!$ faster, despite long attention [16, 17, 18].

A second face: $(\mathbb{Z}/2^m \mathbb{Z})^* = \langle -1 \rangle \times \langle 3 \rangle$, so $\prod_{k \leq K} (1 + 8k) \equiv \pm 3^E \pmod{2^m}$ where

$$E = \sum_{k \leq K} \text{dlog}_3(1 + 8k) \pmod{2^{m-2}}, \quad m = \Theta(N).$$

Computing the *single integer* E is thus a distillation of the multiplicative side of the barrier — and it falls (Theorem 18 below). Expanding $\text{dlog}_3(1 + 8k) = \log_2(1 + 8k)/\log_2 3$ through the 2-adic logarithm turns E into $\sum_j c_j S_j(K)$ with power sums $S_j(K) = \sum_{k \leq K} k^j$; Faulhaber’s formula then prices the exchange in Bernoulli numbers, and the coefficient mass of every arrangement we tried is $\Theta(N^2)$ bits. Third face: merge depth — summing T inhomogeneous word-size rationals below $M(\text{total}) \log T$.

A fourth face is analytic, and rests on the observation that Euler–Maclaurin summation, divergent-asymptotic over $\mathbb{R}$, is *exactly convergent* over $\mathbb{Z}_2$ for our summand.

Lemma 16 (2-adic Euler–Maclaurin). *Let $f(x) = \text{Log}_2(1 + 8x)$ (the 2-adic logarithm) and $F(x) = ((1 + 8x)(f(x) - 1) + 1)/8$. Then, as an identity in $\mathbb{Z}_2$ with the right side absolutely convergent (the i -th correction has 2-adic valuation $\geq 6i - 8$),*

$$\sum_{k=1}^K f(k) = F(K) + \frac{1}{2}f(K) + \sum_{i \geq 1} \frac{B_{2i}}{(2i)!} \left(f^{(2i-1)}(K) - f^{(2i-1)}(0) \right),$$

where $f^{(r)}(x) = (-1)^{r-1}(r-1)!8^r(1+8x)^{-r}$. (*Machine-validated exactly at $(K, m) = (100, 160)$ and $(257, 224)$ 2-adic bits.*)

Since the correction series is, up to elementary factors, the Stirling tail $\sum_i B_{2i} w^{2i-1}/(2i(2i-1))$ evaluated at $w = 8/(1+8K)$ (valuation 3), the lemma says: E — hence $\prod_{k \leq K} (1+8k) \bmod 2^m$, hence the barrier — is a single-point evaluation of Diamond’s 2-adic log-gamma function [24] in its domain of convergent asymptotics, plus elementary terms; and conversely, such evaluations at arguments $8/(1+8K)$, $K = \Theta(m)$, reduce back to E -type sums by reading the identity in reverse. Choosing $K = 2^s$ and running the dyadic doubling recursion instead reproduces partial 2-adic Hurwitz zeta values at positive integers, confirming the same equivalence from the Iwasawa side (cf. [25]). Every known evaluation strategy for these p -adic special functions pays the tapered-Bernoulli mass $\Theta(N^2)$ bits or falls back to the product tree — but the reduction imports the problem into computational Iwasawa theory, where evaluation of p -adic L -functions is an active algorithmic topic, and progress there now transfers here.

A fifth face makes the *input* side provably cheap. The counting measure μ on our progression restricted to $[0, 2^t)$ has Amice transform

$$A_\mu(T) = \sum_{\substack{u < 2^t \\ u \equiv 1 \pmod{8}}} (1+T)^u = (1+T) \frac{(1+T)^{2^t} - 1}{(1+T)^8 - 1},$$

sparse and rational — t squarings — and E is the *first Taylor coefficient* of the Leopoldt Γ -transform (the pushforward of μ along dlog_9) of this nearly-free object; both statements are machine-verified. The Γ -transform is the central computation of Iwasawa theory, and every known way of performing it pays the quadratic mass. Moreover, tracing the state of the dyadic-doubling computation shows that at every scale the true state is the node set — $O(N \log N)$ bits — while the moment, Bernoulli, inverse-power-sum and Iwasawa-coefficient representations that carry the $\Theta(N^2)$ mass are redundant expansions of it: *no entropy or output-size argument can defend the barrier at any intermediate point.* The mass is a property of every known basis, not of the problem. We note the archimedean contrast: the corresponding real object — a Stirling tail at one point — is computable in quasi-linear time via the Binet integral and double-exponential quadrature, and the 2-adic twin resists precisely for lack of a p -adic fast quadrature. This lack is structural, not an accident of ignorance: translation-invariance makes the Volkenborn/Riemann-sum scheme the *canonical* 2-adic quadrature — its Euler–Maclaurin acceleration *is* the Bernoulli mass, Bernoulli numbers being themselves the Volkenborn integrals of monomials — whereas over $\mathbb{R}$ the freedom to deform contours and measures is exactly what powers fast quadrature. The only 2-adic deformation freedom is the choice of measure, which is the Iwasawa-theoretic route already accounted above.

Two further faces close the circle. Sixth: the ψ -operator of Coleman theory intertwines with our dyadic doubling (the measure is Frobenius-structured, A_μ being a ratio of iterates of $\varphi(T) = T^2 + 2T$), and exhibits μ as a level- t truncation of the Mazur measure whose Γ -transform is the Kubota–Leopoldt 2-adic zeta function: computing E fast is thereby *the same problem* as computing ζ_2 to high 2-adic precision, an independently open problem of computational Iwasawa theory. Seventh, and most elementary: since $\mathrm{Log}_2 u = \lim_r (u^{2^r} - 1)/2^r$, one has the machine-validated congruence

$$\sum_{k \leq K} (1+8k)^{2^r} \equiv K + 2^r \sum_{k \leq K} \mathrm{Log}_2(1+8k) \pmod{2^{2r+5}},$$

so the entire barrier is equivalent to a statement requiring no p -adic language at all: *compute $\sum_{k \leq K} (1 + 8k)^{2^r} \bmod 2^{2^r}$, with $K, r = \Theta(N)$, in $\tilde{O}(N)$ time.*

Remark 17 (the analytic root, and why the exits are closed). The recurring Bernoulli mass has a classical cause: the generating function $x/(e^x - 1)$ has infinitely many poles ($2\pi ik$, the poles of ζ), hence is *not holonomic*, so the Bernoulli sequence satisfies no polynomial recurrence — and every fast point-evaluation technology for series (binary splitting of P -recurrences, bit-burst — including its 2-adic version, which achieves quasi-linear precision for exactly the holonomic class [20]) requires such a recurrence; moreover no bit-burst analogue for the full differentially algebraic class can exist, since every polynomial-time-computable real is a value of a polynomial-ODE solution [21] and the time hierarchy theorem forbids universal quasi-linear evaluation. Over $\mathbb{R}$ this is harmless because integral representations route around the coefficients; over $\mathbb{Z}_2$ the quadrature rigidity above closes that road. The wall is thus the confluence of two facts: *no recurrence, and no integral end-run*. Nor can the formula be shopped away: a series whose denominators were purely geometric would sum to a rational number, so transcendence forces the polynomial factor in every BBP-type denominator, which forces moduli sweeping arithmetic progressions, which forces the faces above. Two further resources are accounted for: quantum period-finding offers nothing, since the per-element task (one 2-adic discrete logarithm) is already classically easy and exact summation to $2^{-\Theta(N)}$ defeats amplitude-estimation; and the mass is depth-polylogarithmic parallelizable, so at scale it is a cost in work, not in wall-clock time.

6.3 A crack: the multiplicative faces fall

Theorem 18 (sub-product-tree evaluation of the multiplicative faces). $\prod_{k \leq K} (8k+1) \bmod 2^m$ — and hence E , the power sums $\sum_{k \leq K} (1 + 8k)^{2^r} \bmod 2^{2^r}$, and the log-gamma values of Lemma 16 — are computable in $O(M(N) \log N) = O(N \log^2 N)$ bit operations.

Proof sketch. Borwein’s factorial method [16] transfers. The exponent of an odd prime p in the product is $e_p = \sum_j \#\{k \leq K : p^j \mid 8k + 1\}$, a Legendre-type count on an arithmetic progression, $O(1)$ per prime power after a sieve of cost $O(N \log \log N)$. The distinct-prime mass is $\sum_{p \leq 8K} \lg p = \Theta(N)$ bits: the factored representation compresses the $\Theta(N \log N)$ -bit term list to $\Theta(N)$ bits, with multiplicities riding as exponents. Assemble $\prod p^{e_p} \bmod 2^m$ by exponent-bit scheduling: per bit level, one truncated product tree over the participating primes (level masses decay geometrically from $\Theta(N)$; total $O(M(N) \log N)$) and one squaring mod 2^m ($O(\log N)$ squarings in all). Finally $E = \text{Log}_2(\prod) / \text{Log}_2 9$, the 2-adic logarithm being quasi-linear in precision [20]. Machine-validated at six sizes through $K = 3 \cdot 10^6$: exact agreement with the direct product, with the measured time ratio growing monotonically in the factored method’s favor — the log-factor signature. $\square$

Three consequences. First, the seven faces are *not* constructively equivalent: they split into a multiplicative cluster — products, E , power sums, log-gamma/truncated Kubota–Leopoldt values, now all $O(N \log^2 N)$, which in particular is the first recorded sub-product-tree evaluation of Diamond’s log-gamma at such arguments (no p -adic Gamma routine exists in the standard libraries at all) — and the *additive core*: V itself, the consolidated sum $\sum_t \pm 2^{J_{\max} - J_t} \mu_t^{-1}$, whose numerator admits no prime factorization. Second, Theorem 1 is unchanged, since the digit pipeline consumes V : the barrier now stands, sharper, on the additive face alone. Third, a methodological vindication: Conjecture 20 (below) predicted that any crack must leave every basis it names — and this one does, entering through the one representation absent from that list, multiplicities-as-exponents.

6.4 Routes examined

Route	Outcome
lcm-thinned splitting tree	constants only ($\sim 5\times$); mid-tree mass survives
precision-tapered 2-adic trees	reconverge to the product tree
Montgomery batch inversion	quadratic at full precision
2-adic Lerch series / first-order ODE (bit-burst)	holonomic evaluation, same class
Faulhaber–Bernoulli exchange for E	$\Theta(N^2)$ -bit coefficient mass
p -adic Γ at large argument	functional equation unrolls the product
Strassen/BGS baby-step–giant-step mod 2^N	ring-element mass $\Theta(N^2)$
Iwasawa dyadic doubling with inverse-power corrections	initial vector mass $\Theta(N^2)$
2-adic Euler–Maclaurin (Lemma 16)	exact reduction to G_2 -evaluation; mass persists
Newton-series/Abel transform of the V -sum	returns hypergeometric splitting
Γ -transform of the sparse-rational counting measure	input free; the pushforward carries the mass
dyadic doubling as Möbius/Taylor shifts	$\tilde{O}(N^2)$; exposes the operator structure
near-linear series composition [22], checked per face	composition cannot re-index exponents
TC^0/CRT iterated-product circuits [23]	mod- p dlog mirror: full periods free, partials stall
sieve/von Mangoldt restructurings of E	exact rearrangements; the hard core is conserved

The Strassen/BGS entry deserves emphasis: $n! \bmod p$ is computable in $\tilde{O}(\sqrt{n})$ ring operations [17, 18] — sublinear — because the ring elements are single words. At precision $2^{\Theta(N)}$ the same technique carries $\Theta(N)$ -bit elements and the advantage vanishes. The barrier is a statement about *mass*, not about term count.

6.5 The barrier is not cryptographically protected

Fast factorials modulo an *arbitrary* integer m would factor m (Shamir’s classical argument [19]: binary-search $\gcd(k! \bmod m, m)$), which is often cited as evidence that factorials must be hard. That shield does not cover our instance: the modulus is the fixed prime power $2^{\Theta(N)}$, where the gcd argument is vacuous. To our knowledge nothing but the absence of an idea protects $\prod (8k+1) \bmod 2^{\Theta(N)}$, E , or V . We consider this the most attackable formulation and state it as the target:

Open Problem 19. Compute the additive core $V = \sum_t \pm 2^{J_{\max}-J_t} (\mu_t^{-1} \bmod 2^{J_{\max}}) \bmod 2^{J_{\max}}$ — equivalently its numerator $\sum_t \pm 2^{J_{\max}-J_t} \prod_{s \neq t} \mu_s \bmod 2^{J_{\max}}$ — in $O(M(N) \log N)$ bit operations. Via the pipeline of Section 4 this immediately yields decimal (and binary) digit extraction of π in $O(N \log^2 N)$. The denominator product is computable within that budget (Theorem 18); the *sum* is the open half, and no factoring-hardness or information-theoretic argument protects it.

Conjecture 20 (mass conservation, additive form). Every algorithm computing the additive core V of Open Problem 19 by ring operations over $\mathbb{Z}/2^{O(N)}$ whose intermediate values are coefficient arrays in the bases occurring in this section — moments, Bernoulli/Iwasawa coefficients, inverse-power sums, symmetric functions, or per-prime residue systems — reads or writes $\Omega(N^2 / \log^{O(1)} N)$ bits in total. (The original form of this conjecture also covered E ; Theorem 18 resolved that case exactly as the conjecture’s guidance predicted — by an unlisted representation.)

The routes of the table, spanning real-analytic, 2-adic, mod- p and circuit-theoretic bases, are the conjecture’s evidence; the compactness observation above shows it cannot be proven information-theoretically, and an unconditional proof would amount to an explicit superlinear

circuit lower bound, beyond current techniques. Its practical content is guidance: a resolution of Open Problem 19 must operate outside every basis the conjecture names.

7 Implementation and verification

Two v1 implementations accompany this paper. The pure-Python reference `decimal_bbp_extract.py` covers both π and $\ln 2$ and includes an exact per-term auditor and self-tests. The dependency-free C++ port `three_phase_extract.cpp` adds OpenMP, Barrett-reduced Horner over the shorter of the prefix $[0, J)$ or suffix $[J, L)$ per band term, a 96-bit fixed-point accumulator, and internal computation of X by word-multiply passes. The v1 loops realize the naive band reduction, hence carry the $O(N^2/w)$ word-operation term. The batching of Theorem 7 is additionally implemented as a Python reference (`extract_v2`: one accumulating remainder tree over the merged band terms of all four families, with variable chunk widths δ_t , which also covers $\ln 2$'s non-uniform exponents after sorting by J); it reproduces the v1 fixed-point accumulator bit-for-bit at 88 position/constant combinations. The refined pipeline of Section 4 itself is implemented as a third path (`extract_v3`: the Bézout word phase plus the division-free V merge, with symbolic shifts and per-node precision caps, over GMP integers via `gmpy2` when available); it reproduces the v1 digits across the same battery and at every benchmark position below.

Correctness. All of the following were verified against digits computed independently by integer Chudnovsky binary splitting (and, for $\ln 2$, by binary splitting of $2 \operatorname{artanh}(1/3)$): positions 1–60 of π continuously (8-digit windows); π at $N = 10^2, 10^3, 5 \cdot 10^3, 10^4, 10^5, 10^6$; $\ln 2$ at 1–40 and $N = 10^2, 10^3, 10^4, 10^5$; agreement at every tested position between the BBP-16 path and the independent Strassnitzky–Dase path; and an exact-rational audit of 397 random band/tail terms (worst fixed-point deviation $2^{-153.6}$, bound 2^{-142}). At $N = 10^6$ the extractor returns 130927562832 at positions $10^6..10^6+11$, matching the reference. At $N = 10^7$ the C++ implementation returns 725915133612 at positions $10^7..10^7+11$; we verified this window against two independently published digit corpora — Google's 100-trillion-digit computation (served at `pi.delivery`, with the index convention confirmed by probing the leading digits) and the 200-million-digit corpus at `angio.net`, whose substring search locates the window at exactly position 10^7 after the decimal point.

Identity-level validations. The mechanisms of Sections 4–6 are backed by exact machine checks: the chain/CRT mechanism of Theorem 10 reproduces $2^{4k} \bmod (8k+1)$ for all $k \leq 2 \cdot 10^4$ (with the Euler-criterion consequence $2^{4k} \equiv 1$ for prime $8k+1$ confirmed throughout, and $2.7 \times$ fewer word multiplications than per-term square-and-multiply at that size); Lemma 12 was verified exactly on 300 random band terms; Lemma 13 was verified exactly against the termwise dyadic sum; Lemma 16 was verified exactly at $(K, m) = (100, 160)$ and $(257, 224)$; the fifth-face Amice form and the extraction $E = \operatorname{Log}(\prod u) / \operatorname{Log} 9$ were verified exactly at 96 bits; and the seventh-face congruence was verified exactly at $(K, r) = (200, 120)$ — all 245 bits — including recovery of the Log-sum from the elementary power sum alone.

Timings (Windows 11 desktop, 16 hardware threads; C++ with OpenMP; Python single-thread, CPython 3.10):

N	Python v1	C++ v1	auxiliary string
10^4	0.12 s	0.006 s	2.9 KB
10^5	6.4 s	0.33 s	29 KB
10^6	571 s	32.3 s	290 KB
10^7	—	3421 s	2.9 MB

For v1 the measured exponent bends from $N^{1.67}$ at small sizes to $N^{2.03}$ over the decade 10^6 – 10^7 , as predicted for the naive band reduction. For an external baseline we compiled the verification-grade implementation of Gourdon’s method — the strongest published competitor — and raced it on the same machine: it measures 0.59 s, 32.3 s and 2069 s at $N = 10^4, 10^5, 10^6$ (its digits agreeing with our verified windows throughout), so the refined pipeline below is respectively $10\times$, $42\times$ and $232\times$ faster, with the gap growing without bound. The refined pipeline settles the practical question:

N	v1 Python	v1 C++ (16 threads)	v3 Python (1 thread, GMP)
10^4	0.12 s	0.006 s	0.06 s
10^5	6.4 s	0.33 s	0.77 s
10^6	571 s	32.3 s	8.9 s
10^7	—	3421 s	111.6 s

The v3 exponent fitted over 10^4 – 10^7 — three decades — is $N^{1.077}$, with no upward bend; the quasi-linear regime of Theorem 1 is thus visible directly, in interpreted code. At $N = 10^7$ the single-threaded v3 outperforms the 16-thread compiled v1 by $30.7\times$ (and returns the corpus-verified window 725915133612); against a hypothetical quadratic continuation of v1’s own small- N constants, the gap at 10^7 exceeds two orders of magnitude. The v3 phase split at 10^7 (64 s exponentiation loop, 47 s for the entire V phase, 0.05 s to build X) confirms that the plain word loop is now the dominant cost — precisely the phase that the sieve-batching of Theorem 10 addresses. A pure-Python integration of that batching (v4) validates the mechanism at scale — identical digits through $N = 10^7$, with the chain/CRT machinery processing $3.3 \cdot 10^6$ terms — but is $1.35\times$ *slower* there in wall clock: the amortization’s gain is in multiplication count, which interpreted per-statement overhead conceals (≈ 8 – 10 interpreted operations per chain hit against a single C-level exponentiation call). Realizing Theorem 10 in running time, as opposed to operation count, therefore awaits the compiled port, for which v4 serves as the correctness oracle.

8 Open problems

(1) Integrate the sieve-batching of Theorem 10 into the v3 pipeline — its word loop is now the measured bottleneck — and port to C+GMP for record-scale positions. (2) Open Problem 19: the one remaining logarithm, in any of its four faces. (3) Decimal digit extraction in sublinear space — the original problem of [1] — remains open; is there a lower-bound obstruction linking it to the complexity of isolated middle bits of 5^n ? (4) The per-term independence invites a distributed record attempt: verifying isolated *decimal* positions of record computations of π (a practical need noted by record holders) requires exactly this primitive. (5) Zudilin’s Gaussian-integer series can presumably be treated by the same band identity over $\mathbb{Z}[i]$ with a shared

string for $(1-2i)^d$; carrying this out would give a second, structurally different quasi-linear base-5 extractor.

Acknowledgments

The author thanks the maintainers of the computational records and surveys cited here.

References

- [1] D. Bailey, P. Borwein, S. Plouffe, *On the rapid computation of various polylogarithmic constants*, Math. Comp. **66** (1997), 903–913.
- [2] D. Borwein, J. M. Borwein, W. F. Galway, *Finding and excluding b-ary Machin-type individual digit formulae*, Canad. J. Math. **56** (2004), 897–925.
- [3] S. Plouffe, *On the computation of the n 'th decimal digit of various transcendental numbers*, 1996; arXiv:0912.0303.
- [4] F. Bellard, *Computation of the n 'th digit of π in any base in $O(n^2)$* , 1997, https://bellard.org/pi/pi_n2/pi_n2.html.
- [5] X. Gourdon, *Computation of the n -th decimal digit of π with low memory*, unpublished manuscript, February 11, 2003, <http://numbers.computation.free.fr/Constants/Algorithms/nthdecimaldigit.pdf>.
- [6] S. Plouffe, *A formula for the n 'th decimal digit or binary of π and powers of π* , arXiv:2201.12601 (2022).
- [7] W. Zudilin, *A BBP-style computation for π in base 5*, arXiv:2409.10097 (2024). (v1 titled “...in base 10”.)
- [8] D. H. Bailey, *A compendium of BBP-type formulas for mathematical constants*, manuscript, updated 2023, <https://www.davidhbailey.com/dhbpapers/bbp-formulas.pdf>.
- [9] A. Borodin, R. Moenck, *Fast modular transforms*, J. Comput. System Sci. **8** (1974), 366–386.
- [10] D. J. Bernstein, *Scaled remainder trees*, manuscript, 2004, <https://cr.yp.to/arith/scaledmod-20040820.pdf>.
- [11] E. Costa, R. Gerbicz, D. Harvey, *A search for Wilson primes*, Math. Comp. **83** (2014), 3071–3091.
- [12] D. Harvey, A. V. Sutherland, *Computing Hasse–Witt matrices of hyperelliptic curves in average polynomial time*, LMS J. Comput. Math. **17** (2014), 257–273.
- [13] D. Harvey, J. van der Hoeven, *Integer multiplication in time $O(n \log n)$* , Ann. of Math. **193** (2021), 563–617.
- [14] P. Afshani, C. B. Freksen, L. Kamma, K. G. Larsen, *Lower bounds for multiplication via network coding*, in: ICALP 2019, LIPIcs **132**, 10:1–10:12.

- [15] D. Zeilberger, W. Zudilin, *The irrationality measure of π is at most 7.103205334137...*, Exp. Math. **24** (2015), 419–423.
- [16] P. B. Borwein, *On the complexity of calculating factorials*, J. Algorithms **6** (1985), 376–380.
- [17] V. Strassen, *Einige Resultate über Berechnungskomplexität*, Jahresber. Deutsch. Math.-Verein. **78** (1976/77), 1–8.
- [18] A. Bostan, P. Gaudry, É. Schost, *Linear recurrences with polynomial coefficients and application to integer factorization and Cartier–Manin operator*, SIAM J. Comput. **36** (2007), 1777–1806.
- [19] A. Shamir, *Factoring numbers in $O(\log n)$ arithmetic steps*, Inform. Process. Lett. **8** (1979), 28–31.
- [20] X. Caruso, M. Mezzarobba, N. Takayama, T. Vaccon, *Fast evaluation of some p -adic transcendental functions*, in: ISSAC 2021; arXiv:2106.09315.
- [21] O. Bournez, D. S. Graça, A. Pouly, *Polynomial time corresponds to solutions of polynomial ordinary differential equations of polynomial length*, J. ACM **64** (2017), no. 6, art. 38.
- [22] Y. Kinoshita, B. Li, *Power series composition in near-linear time*, in: FOCS 2024.
- [23] W. Hesse, E. Allender, D. A. Mix Barrington, *Uniform constant-depth threshold circuits for division and iterated multiplication*, J. Comput. System Sci. **65** (2002), 695–716.
- [24] J. Diamond, *The p -adic log gamma function and p -adic Euler constants*, Trans. Amer. Math. Soc. **233** (1977), 321–337.
- [25] L. C. Washington, *Introduction to Cyclotomic Fields*, 2nd ed., GTM 83, Springer, 1997.